

A Multi-Level Autonomous Delivery Robot System in Unity for Construction-Site Material Transportation

Jingxuan Duan
Independent Study

shureduan0912@gmail.com

Abstract

Construction-site material logistics is a labor-intensive and schedule-critical task. Nighttime idle windows on active sites offer an opportunity for autonomous robot-based transport without interfering with daytime operations. We present a full-stack simulation system for multi-floor autonomous delivery in a Unity-based construction-site environment. The system integrates four tightly coupled layers: (i) per-floor occupancy maps obtained through simulated LiDAR SLAM; (ii) a topological node-edge graph spanning three vertical levels, connected by ramps and an elevator; (iii) a combined global route planner that uses Theta for continuous-space path planning on each floor and a Capacitated Vehicle Routing Problem (CVRP) solver based on the CBC Mixed-Integer Linear Program (MIP) for multi-trip delivery scheduling; and (iv) a Unity-side Pure Pursuit controller that executes the resulting continuous waypoint stream, including cross-floor elevator transitions. An additional local robustness layer—PARS (Path-Attentive Recovery SAC), a Soft Actor-Critic (SAC)-based recovery policy—is designed and partially implemented to handle dynamic obstacles that the global planner cannot foresee. We validate the end-to-end pipeline through simulation experiments with randomized delivery targets and obstacle configurations. The proposed system demonstrates a feasible architecture for autonomous nighttime material delivery in complex, multi-floor construction environments.*

1. Introduction

1.1. Motivation

Material transportation is a persistent and quantifiable bottleneck in large-scale construction projects. Survey evidence from Tunji-Olayeni et al. identifies “transport” as the single most severe logistics challenge on construction sites, with a mean severity score of 4.45 out of 5—above storage inadequacy (4.18) and delivery delay (4.13) [1]. The inefficiency operates at multiple levels. At the equipment level, every

Table 1. Empirical evidence of construction logistics inefficiency.

Issue	Key Evidence	Category
Transport waiting	Severity 4.45/5, ranked #1 [1]	Transport
Worker idle time	10–12% labour cost loss [3]	Productivity
Crane cycle waiting	Systematic across all phases [2]	Transport
Delivery failure	<50% arrive on time & correct [4]	Delivery Error
Night-time idle	8–12 h/day site inactive; 0 autonomous deliveries	Night-time
Robot potential	20% faster; 60–90% labour reduction [5]	Automation

phase of a tower crane cycle—approach, attach, lift, transport, lower, detach—generates systematic operator-waiting idle time traceable through on-board data loggers [2]. At the supply chain level, fragmented delivery scheduling forces workers to stand idle awaiting material arrivals; eliminating this waiting alone has been estimated to recover **10–12% of total labour cost** on road construction projects [3]. Even when deliveries do arrive, fewer than half reach the correct location, in the correct quantity, undamaged, and on time [4]. Table 1 summarises the empirical evidence.

Compounding this daytime inefficiency is a structural under-utilisation of nighttime hours. Large construction sites are substantially idle for 8–12 hours per day, yet material pre-positioning for the next day is typically left to the morning rush, creating a predictable bottleneck at every shift start. Consolidating deliveries into off-peak or nighttime windows reduces delivery frequency and on-site disruption, a benefit demonstrated on large urban construction projects [4]. Autonomous delivery robots are natural candidates for this role: they operate without fatigue, can navigate without lighting, and follow exactly optimised delivery sequences without crew coordination overhead [6]. Industry projections estimate that robotic material transport can accelerate project timelines by **20%** and reduce target-task labour demand by

60–90% [5].

1.2. Technological Readiness

The conditions necessary to realise autonomous construction delivery are now falling into place. Modern large-scale construction sites routinely deploy building information modelling (BIM), on-site UWB/GPS localisation networks, and IoT-based material tracking systems—exactly the digital map and live demand data that a delivery robot requires [7]. On the platform side, the core mobile-robot stack (LiDAR SLAM, onboard edge compute, and reliable long-duration wheeled locomotion) has been validated at industrial scale in structurally analogous environments: coordinated fleets of hundreds of autonomous mobile robots already handle continuous multi-trip material transport in large-scale fulfilment centres without human supervision [8], and recent reviews confirm their expanding footprint in the built environment [6]. Looking forward, falling sensor costs, advances in learned obstacle avoidance, and rising industry adoption of robotic platforms on active sites define a near-term window in which autonomous construction delivery can transition from research demonstration to operational practice—with projected gains of 20% in project completion speed and 60–90% reduction in targeted labour demand [5] providing clear economic justification.

1.3. Challenges

Adapting autonomous-driving or warehouse-robotics techniques directly to construction sites is non-trivial for three reasons. *Multi-floor topology*: goods must travel between ground level, elevated platforms, and upper floors via ramps and elevators, requiring systems that reason about vertical connectivity. *Dynamic obstacles*: workers, parked machinery, and material piles create irregular, time-varying obstacle fields that differ from structured warehouse aisles or open roads. *Heterogeneous multi-trip demand*: multiple cargo types must be scheduled across delivery trips under vehicle capacity constraints, while the robot repeatedly returns to a depot—a problem structure that standard single-trip CVRP formulations cannot represent faithfully.

1.4. Contributions

This work presents an end-to-end autonomous delivery robot system for multi-floor construction sites, with two principal contributions.

1. Explicit multi-trip multi-commodity CVRP formulation. Standard CVRP assumes a single route per vehicle. Construction logistics requires repeated depot returns within one planning horizon (the robot reloads and redispaches multiple times). We introduce trip-indexed route variables $x_{k,i,j}$ and explicit unloading variables $\delta_{k,c,g}$, coupled with multi-commodity flow conservation and tight bounds on

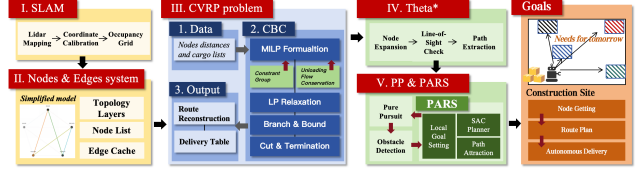


Figure 1. System pipeline overview.

the number of active trips $[K_{\min}, K_{\max}]$. This formulation (i) guarantees physical consistency (no goods delivered without prior depot loading), (ii) tightens the MIP feasible region, and (iii) directly models the repeated loading–transport–unloading cycle characteristic of construction-site robot logistics.

2. Integrated multi-floor topological delivery system.

Existing work addresses either task-level scheduling (warehouse-style order optimisation) or robot navigation (single-floor path following) in isolation. We unify construction-site dynamic material demand, vehicle capacity constraints, multi-floor topological structure with multiple entry/exit transfer points, and local path feasibility into a single hierarchical delivery decision system. Concretely: a CVRP/task-allocation layer decides *what to deliver and in what order*; a multi-floor topological graph with Theta*-planned intra-floor paths and elevator transfer edges converts this abstract sequence into executable cross-floor waypoints; and a hybrid Pure-Pursuit–SAC execution layer provides both nominal tracking and learned local obstacle recovery. The system thereby evaluates how changes in site layout and dynamic demand affect delivery decisions and execution performance—a capability that neither order-optimisation nor navigation systems alone can provide.

Beyond these two innovations, we contribute:

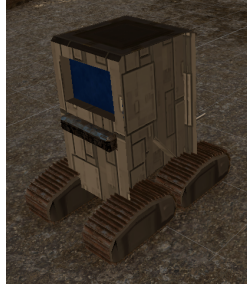
- a SLAM-based per-floor occupancy mapping pipeline for multi-height LiDAR data;
- PARS (Path-Attentive Recovery SAC), a Soft Actor-Critic local recovery module that activates only when Pure Pursuit encounters blockages and rejoins the global plan without replanning;
- full end-to-end validation in a high-fidelity Unity construction-site simulation across 100 randomised deliveries.

1.5. System Overview

Figure 1 illustrates the five-stage pipeline: SLAM-based mapping → topological graph construction → Theta*+CVRP route planning → waypoint generation → Pure Pursuit + PARS execution. The remainder of the paper describes each stage in turn.



(a) Aerial view of the Unity scene.



(b) Delivery robot platform.

Figure 2. Construction-site scene and delivery robot.

2. System Framework

We implement the system in Unity, which provides physics-based collision detection, tight integration of sensor generation and robot control within a single simulation loop, and rendering fidelity that supports future sim-to-real transfer [9].

The scene spans approximately 100×100 m across three vertical levels: **Floor 0** (ground)—depot and ramp access points; **Floor 1** (platform)—elevated working area with delivery regions A and B and the elevator boarding point; **Floor 2**—upper-level area with delivery region C and the elevator exit. Two ramps connect Floor 0 to Floor 1; a single elevator connects Floor 1 to Floor 2. The robot is a wheeled rigid body driven by continuous velocity commands and equipped with a simulated multi-height 2-D LiDAR sensor.

At the start of each experiment, a destination generator places three delivery regions at randomised positions: regions A and B on Floor 1 at least 12 m apart, and region C on Floor 2. The Python planning pipeline reads exported node positions and occupancy maps, solves the routing problem offline, and writes the resulting waypoint stream to a CSV file that the Unity route executor loads and dispatches to the controller.

3. Topological Environment Modeling and Route Planning

3.1. Per-Floor Occupancy Mapping

Because the site spans three vertically disconnected levels, a single global occupancy map would conflate obstacles from different floors. We generate an independent 2-D occupancy grid per floor by teleoperated coverage scanning with a multi-height LiDAR; range measurements are accumulated into a world-frame probability grid using the robot’s broadcast pose. Two production maps are generated: one for the platform (Floor 1), reused as the PARS training map, and one for Floor 2. Figure 3 shows the platform map.

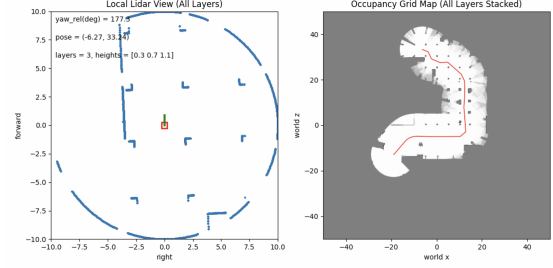


Figure 3. Platform-floor occupancy map from simulated LiDAR.

3.2. Multi-Floor Topological Graph

Rather than planning on the dense occupancy grid for the full multi-floor route, we build a sparse topological graph over semantically meaningful positions. Ten nodes are defined across the three floors: the ground depot, two ramp-bottom and two ramp-top nodes, the elevator boarding and exit nodes, and the three delivery-region nodes (A, B on Floor 1; C on Floor 2).

Edges fall into three categories. *Fixed* edges encode pre-surveyed corridors and ramp segments as three-dimensional waypoint sequences. *Theta* edges are intra-floor paths computed automatically by the planner on the occupancy grid. *Transfer* edges represent elevator transitions, storing only the boarding and alighting positions. All edges are cached in a unified graph; fixed and transfer edges are populated at construction time, theta edges are filled in after Theta* runs.

3.3. Theta*-Based Path Planning

We use Theta* [10], an any-angle variant of A* that allows parent pointers to skip intermediate grid nodes via line-of-sight, producing shorter and smoother paths than grid A*.

Each map is preprocessed by binarizing occupancy, inflating obstacles by the robot radius (0.8 m), and computing a Euclidean clearance map. The Theta* parent-update rule is:

$$g_{\text{new}}(s') = \begin{cases} g(p(s)) + \|p(s) - s'\|_2 & \text{if LoS}(p(s), s'), \\ g(s) + \|s - s'\|_2 & \text{otherwise.} \end{cases} \quad (1)$$

Theta* is applied to six topology edges (five on Floor 1, one on Floor 2); Figure 4 shows the resulting paths.

3.4. Multi-Trip Multi-Commodity CVRP

Problem and standard limitation. Given the inter-node distance matrix from Dijkstra over the merged graph, we schedule deliveries as a multi-commodity CVRP [11] solved with the CBC MIP solver [12]. Standard CVRP assumes each vehicle completes a single route. On a construction site, however, the robot must repeatedly return to the depot to reload before continuing—a repeated depot-return structure that single-trip formulations cannot model. If this is handled

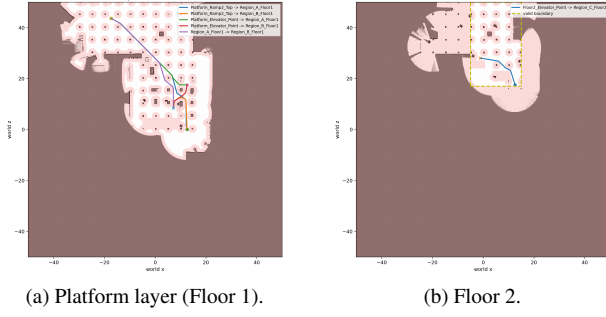


Figure 4. Theta* paths on per-floor occupancy maps.

implicitly, the solver may produce routes where goods appear to be delivered without ever being loaded, violating physical feasibility.

Proposed formulation. To address this, the present work introduces the following variables specifically designed for repeated-return construction-site delivery:

- *Trip-indexed route variables* $x_{k,i,j} \in \{0, 1\}$ and activity indicator $y_k \in \{0, 1\}$, allowing the solver to explicitly distinguish each individual delivery trip made by the robot.
- *Explicit unloading variables* $\delta_{k,c,g} \in \mathbb{Z}_{\geq 0}$, which directly represent the quantity of good g delivered to customer c in trip k , connecting routing decisions to physical delivery events.
- *Bounded trip count* $[K_{\min}, K_{\max}]$, preventing unbounded search while preserving flexibility for fragmented multi-floor delivery:

$$K_{\min} = \left\lceil \frac{\sum_{c,g} D_{c,g}}{Q} \right\rceil, \quad K_{\max} = K_{\min} + 2|\mathcal{C}|. \quad (2)$$

- *Flow conservation constraints* (Section 6) that close the full delivery loop: depot loading $\rightarrow$ route transport $\rightarrow$ customer unloading, ensuring every delivery is backed by a corresponding depot departure.

The combined effect is a tighter MIP feasible region that eliminates physically inconsistent solutions, and a formulation that directly captures the load–transport–unload–return cycle unique to construction-site robot logistics. Multi-commodity flow variables $f_{k,i,j,g}$ track per-good flows along each arc.

Objective. Minimise total travel distance:

$$\min \sum_{k,i \neq j} d_{ij} x_{k,i,j}. \quad (3)$$

Constraints. Routing structure (depot and customer degree):

$$\sum_{j \neq 0} x_{k,0,j} = y_k = \sum_{i \neq 0} x_{k,i,0}, \quad \sum_{j \neq c} x_{k,c,j} = v_{k,c} = \sum_{i \neq c} x_{k,i,c}. \quad (4)$$



Figure 5. Executable waypoint stream in the Unity scene.

Capacity, flow conservation, and demand fulfilment:

$$\sum_g f_{k,i,j,g} \leq Q x_{k,i,j}, \quad \sum_{c,g} \delta_{k,c,g} \leq Q y_k, \quad (5)$$

$$\sum_{i \neq c} f_{k,i,c,g} - \sum_{j \neq c} f_{k,c,j,g} = \delta_{k,c,g}, \quad \sum_k \delta_{k,c,g} = D_{c,g}. \quad (6)$$

MTZ subtour elimination:

$$u_{k,c} - u_{k,c'} + |\mathcal{C}| x_{k,c,c'} \leq |\mathcal{C}| - 1 + |\mathcal{C}|(1 - v_{k,c'}). \quad (7)$$

The explicit delivery variable $\delta_{k,c,g}$ coupled with flow conservation guarantees physical consistency: no goods are delivered without a corresponding depot loading event, closing the depot-loading $\rightarrow$ transport $\rightarrow$ unloading loop. The CBC solver finds proven optimal solutions for the current four-node instance in under one second.

Route generation. The CVRP trip sequence is expanded into an executable waypoint stream by chaining per-edge waypoints via Dijkstra, inserting dwell commands at elevator and delivery nodes, and writing the result to a flat CSV that the Unity executor reads directly. Figure 5 shows a segment in the Unity scene.

4. Execution and Local Robustness Enhancement

4.1. Pure Pursuit Tracking in Unity

Route execution uses a Unity implementation of the Pure Pursuit algorithm [13], which steers the robot toward a *lookahead point* on the reference path at a fixed arc-length distance $L_d = 1.2$ m ahead. Given the lookahead point (x_L, z_L) and the robot's heading ψ , the signed heading error is

$$\alpha = \text{atan2}(x_L - x_r, z_L - z_r) - \psi, \quad (8)$$

and the required steering curvature is

$$\kappa = \frac{2 \sin \alpha}{L_d}. \quad (9)$$

A turn-then-go strategy rotates the robot in place before translating when $|\alpha|$ exceeds a threshold, reducing overshoot on sharp bends. A waypoint is marked reached within 0.5 m; the robot hard-stops at the final waypoint.

Critical waypoint locking. The two elevator boarding positions are designated *critical*: on approach the controller switches to a slow point-to-point mode with yaw-only correction to guarantee precise alignment. The route executor also pauses at elevator and delivery nodes to simulate dwell-and-unload behaviour.

4.2. Cross-Floor Execution Logic

The merged edge graph contains three edge types handled differently at runtime. *Fixed* edges encode pre-surveyed corridors and ramp segments with three-dimensional waypoints; Pure Pursuit climbs or descends ramps naturally as the waypoints carry correct vertical heights. *Theta* edges are intra-floor paths on the occupancy maps; the executor selects the correct map layer for PARS obstacle checking. *Transfer* edges represent elevator transitions carrying only the boarding and alighting positions; vertical motion is handled by the Unity trigger.

The robot is guided to the boarding waypoint via critical-waypoint locking, then a dwell command confirms elevator activation. After the platform reaches the destination floor, waypoint tracking resumes from the alighting node. The route builder resolves all transfer edges at construction time, so the executor processes the route as a flat sequence with no runtime branching.

4.3. Local RL for Robust Navigation

4.3.1. Motivation and Architecture

The global planner operates on a static map captured at mission start. Workers, equipment, or stockpiles that appear on the planned path cause Pure Pursuit to stall or collide. Global map-based replanning adds hundreds of milliseconds of latency and is disproportionate for short-lived blockages.

We introduce **PARS (Path-Attentive Recovery SAC)**, a Soft Actor-Critic policy [14] for learned local obstacle avoidance. PARS activates only when Pure Pursuit becomes unsafe, and returns control once safe reconnection is geometrically feasible. Compared with graph replanning, PARS (i) preserves the global plan and CVRP trip sequence, (ii) reacts within one control cycle ($\Delta t = 0.2$ s), (iii) outputs continuous velocity commands directly, and (iv) generalises to unseen obstacle layouts through SAC’s maximum-entropy exploration.

4.3.2. State, Action, and Reward

State. The 29-dimensional observation is

$$\mathbf{o}_t = \left[\underbrace{\mathbf{L}_t}_{21}, \underbrace{\mathbf{g}_t^{\text{rob}}}_{2}, \underbrace{\mathbf{w}_{\text{ret},t}^{\text{rob}}}_{2}, e_{\perp}, \Delta\psi, v_t, \omega_t \right], \quad (10)$$

Algorithm 1 PARS: Path-Attentive Recovery SAC

Require: Path $\mathcal{P}$, map $\mathcal{M}$, policy π_{θ}

```

1: mode  $\leftarrow$  PUREPURSUIT
2: while waypoints remaining do
3:   Observe state; compute  $e_{\perp}, \Delta\psi, \mathbf{L}_t$ , projection  $s^*$ 
4:    $\mathbf{w}_{\text{ret}} \leftarrow \mathcal{P}(s^* + \Delta s_r)$ 
5:   if mode = PUREPURSUIT then
6:      $\mathbf{a} \leftarrow \pi_{\text{pp}}(\mathcal{P}, x, z, \psi)$ 
7:     if not LINEFREE( $\mathbf{w}_{\text{ret}}, r_{\text{block}}$ ) then
8:       mode  $\leftarrow$  PARS
9:     end if
10:  else
11:     $\mathbf{a} \leftarrow \pi_{\theta}(\mathbf{o}_t)$ 
12:    if LINEFREE( $\mathbf{w}_{\text{ret}}, r_{\text{rejoin}}$ ) then
13:      Lock rejoin; mode  $\leftarrow$  PUREPURSUIT
14:    end if
15:  end if
16:  Apply  $\mathbf{a}$ ; advance step
17: end while
```

where $\mathbf{L}_t$ is a 180° LiDAR scan (max range 6 m, normalised to $[0, 1]$), $\mathbf{g}_t^{\text{rob}}$ and $\mathbf{w}_{\text{ret},t}^{\text{rob}}$ are the local segment endpoint and forward return waypoint in the robot frame, $e_{\perp}$ is the signed lateral error, $\Delta\psi$ is the heading error, and v_t, ω_t are velocities.

Action. $\mathbf{a}_t = [v_t^{\text{cmd}}, \omega_t^{\text{cmd}}] \in [0, 0.8] \times [-1, 1]$ (m/s, rad/s).

Reward.

$$r_t = w_g \Delta d + w_s \Delta s - w_d f(\mathbf{L}_t) - w_e e_{\perp} - w_a \|\Delta \mathbf{a}_t\| - c_{\text{stall}} + R_{\text{rejoin}} \mathbf{1}(\text{reconnect}). \quad (11)$$

where Δd is goal-distance reduction, Δs is arc-length progress, $f(\mathbf{L}_t) = (\delta - d_{\min})^2 \mathbf{1}(d_{\min} < \delta)$ penalises proximity within safety radius $\delta = 0.8$ m, and c_{stall} fires if displacement is below 1 m in 15 consecutive steps.

4.3.3. Switching Logic

PARS is entered when the inflated line-of-sight check to the forward reference waypoint fails, and exited when the line to the return waypoint clears under a tighter inflation. At exit the rejoin point is locked to prevent target drift. Algorithm 1 states the full execution loop.

4.3.4. Training and Results

PARS is trained in a lightweight Python simulation on the platform occupancy map using six reference routes of varying length. Obstacle count scales with route length (1, 2, or 3 pairs for routes < 10 m, $10\text{--}20$ m, and > 20 m). Training uses SAC with a two-layer MLP (256 units), learning rate 3×10^{-4} , replay buffer 3×10^5 , mini-batch 512, $\gamma = 0.99$,

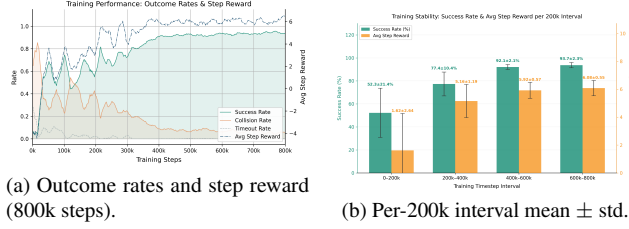


Figure 6. PARS training performance over 800 000 steps.

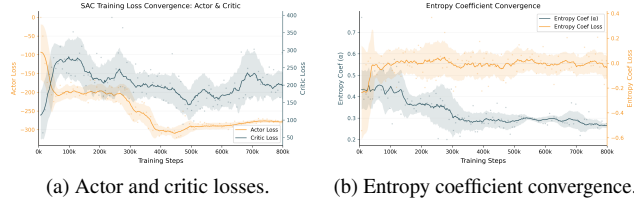


Figure 7. SAC loss convergence and entropy dynamics during PARS training.

Table 2. Qualitative comparison of obstacle-avoidance strategies.

Property	PP only	D* replanning	PP + PARS
Global plan preserved	✓	×	✓
Handles transient obstacles	×	Partial	✓
Requires map update	×	✓	×
Reaction latency	Instant	~100 ms	Instant
Generalises to unseen	×	Limited	✓

$\tau = 0.005$, four parallel environments, and 400 000 total timesteps.

Figure 6 shows the training curves extended to 800 000 steps. The success rate converges above 90% after ~ 280 000 steps and remains stable through 800 000 steps, with per-step reward rising steadily throughout. Figure 6(b) quantifies this stability: mean success rate exceeds 92% and average step reward exceeds 5.9 in the final two 200k-step intervals, with low standard deviation confirming consistent policy performance. Both SAC losses stabilise after ~ 250 000 steps and the entropy coefficient decreases monotonically, confirming progressive policy commitment (Figure 7).

Method comparison. Table 2 summarises key differences between PP-PARS and two baselines.

Avoidance demonstrations. Figure 8 shows two representative PARS trajectories on the platform map; in both cases the policy detours around randomly placed obstacles and rejoins the reference route at the locked rejoin point.

5. Experiments and Conclusion

5.1. Experimental Setup and Results

Scene and procedure. All experiments run in Unity. The scene contains three floors connected by two ramps and one

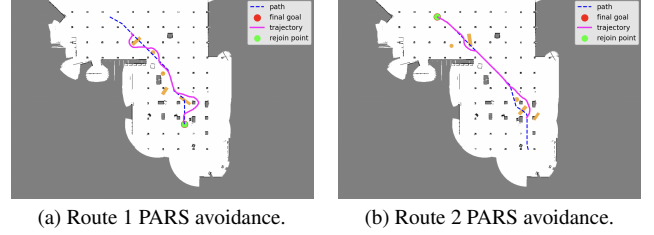


Figure 8. PARS avoidance trajectories (magenta) rejoining the reference path (blue dashed) at the locked rejoin point (green marker).

Table 3. Task completion across five runs (20 deliveries each).

Run	Deliveries	Success Rate	Avg. Trip Time (s)
1	20	1.00	242
2	20	1.00	197
3	20	1.00	261
4	20	1.00	206
5	20	1.00	211
Avg.	20	1.00	223

elevator. At each instance, three delivery regions are placed randomly: two on the platform at least 12 m apart, and one on Floor 2. Each run consists of 20 consecutive deliveries with independently randomised targets. Pure Pursuit tracks the planned route; PARS activates when a randomly placed dynamic obstacle blocks the reference path.

Metrics. We report task success rate (fraction of deliveries completed within the episode budget) and average round-trip time.

Results. Table 3 summarises five runs (20 deliveries each). The system achieved 100% task success across all runs, with a mean round-trip time of 223 s. Elevator transitions completed reliably in every run owing to critical waypoint locking.

Discussion. The results confirm that the integrated pipeline provides stable delivery performance across diverse random configurations. Critical waypoint locking eliminated all elevator misalignment failures observed in pilot runs. The PARS module enabled successful completion of deliveries that a Pure-Pursuit-only controller could not handle, with no additional planning overhead.

5.2. Conclusion and Future Work

We presented an end-to-end autonomous delivery robot system for multi-floor construction-site environments, validated in Unity simulation. The system integrates SLAM-based mapping, a topological graph with three edge types, Theta* intra-floor planning, CBC-based CVRP scheduling, and a hybrid Pure Pursuit-PARS execution layer. Across 100 ran-

domised deliveries the pipeline achieved 100% task success and a 223 s average trip time, demonstrating that the modular architecture is both viable and robust for dynamic construction-site conditions.

Future work will address three directions.

- **Sim-to-real transfer.** Deploying the trained navigation policy on a physical robot requires addressing the sim-to-real gap via domain randomisation and sensor noise augmentation during PARS training.
- **Multi-robot logistics.** The single-robot prototype can be extended to a digital-twin platform that continuously updates maps and schedules, enabling coordinated multi-robot delivery with inter-robot collision constraints.
- **Learned switching criteria.** The current rule-based switching thresholds may reduce robustness in densely cluttered layouts. Replacing them with learned criteria and conducting systematic ablation studies would further improve recovery reliability.

References

- [1] Patience F. Tunji-Olayeni et al. Dataset for material logistics on construction sites. *Data in Brief*, 20:1–6, 2018. 1
- [2] T. Danel, Z. Lafhaj, A. Puppala, S. Lienard, and P. Richard. Proposal for tower crane productivity indicators based on data analysis in the era of Construction 4.0. *Buildings*, 11(1):21, 2021. 1
- [3] Dina M. Mansour, Ibrahim Mahdi, and Ibrahim Abdelrasheed. Impact of material supply chain on the productivity optimization for the construction of roads projects. *Scientific Reports*, 14:3294, 2024. 1
- [4] Pamela C. Nolz. Optimizing construction schedules and material deliveries in city logistics: A case study from the building industry. *Flexible Services and Manufacturing Journal*, 33(3):846–878, 2021. 1
- [5] Thanakon Uthai, Hengxu You, Mengjun Wang, Kaleb Smith, Everett Spackman, Zoe Ryan, Shuai Li, and Jing Du. Opportunities, challenges and roadmap for humanoid robots in construction. *Scientific Reports*, 16:905, 2025. 1, 2
- [6] Lingdong Zeng, Shuai Guo, Jing Wu, and Bernd Markert. Autonomous mobile construction robots in built environment: A comprehensive review. *Developments in the Built Environment*, 19, 2024. 1, 2
- [7] Thomas Bock. The future of construction automation: technological disruption and the upcoming ubiquity of robotics. *Automation in Construction*, 59:113–121, 2015. 2
- [8] Peter R. Wurman, Raffaello D’Andrea, and Mick Mountz. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Magazine*, 29(1):9–20, 2008. 2
- [9] Wenshuai Zhao, Jorge Pena Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: A survey. In *IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020. 3
- [10] Kenny Daniel, Alex Nash, Sven Koenig, and Ariel Felner. Theta*: Any-angle path planning on grids. *Journal of Artificial Intelligence Research*, 39:533–579, 2010. 3
- [11] George B. Dantzig and John H. Ramser. The truck dispatching problem. *Management Science*, 6(1):80–91, 1959. 3
- [12] Google LLC. OR-Tools: Open source software for combinatorial optimization. <https://developers.google.com/optimization>, 2023. Accessed 2026-04-25. 3
- [13] R. Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical Report CMU-RI-TR-92-01, Carnegie Mellon University, Robotics Institute, 1992. 4
- [14] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pages 1861–1870, 2018. 5